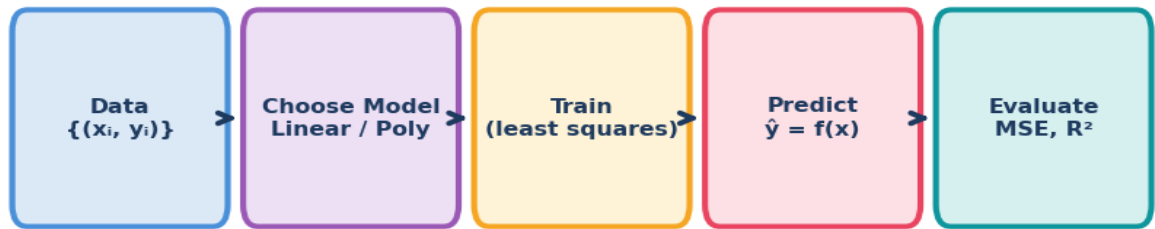


ML Week 2 — Regression Notation Cheat Sheet

Supervised Learning: Linear & Polynomial Regression

The Regression Pipeline

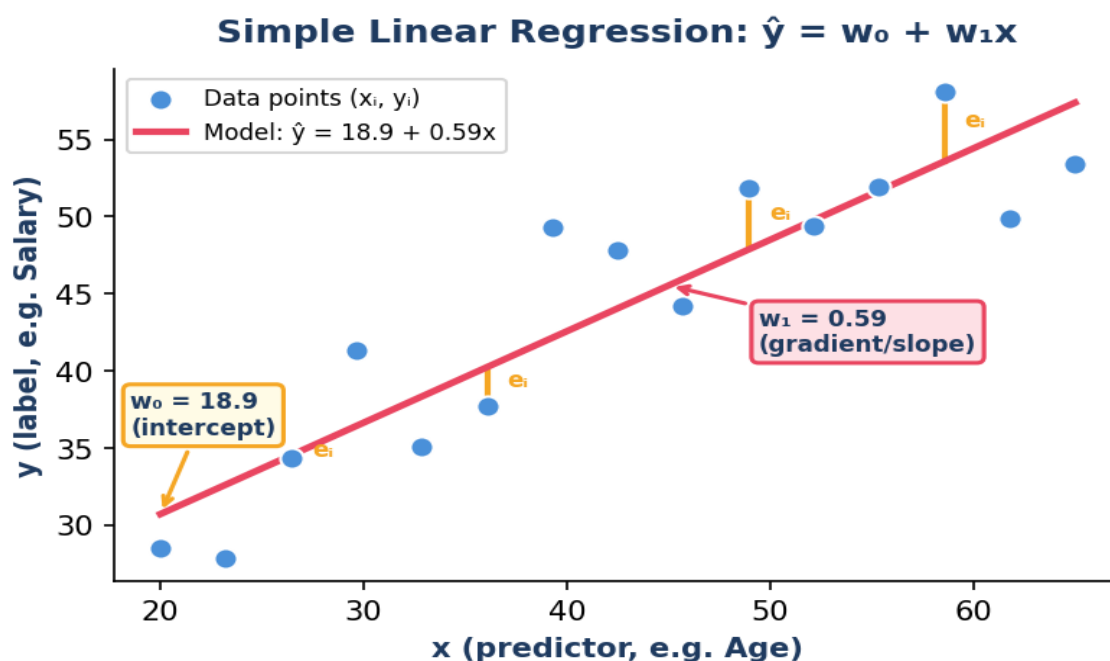


The regression pipeline — collect data, choose model, train (find best w), predict, evaluate.

The Core Symbols

Symbol	Name	Plain English	Think of it as...
x	Predictor(s)	Input features	"What we use to predict"
x_i	Sample i predictors	The i -th data point	"Row i in the spreadsheet"
y	Label (true)	The target value	"The right answer (continuous)"
$\hat{y} = f(x)$	Prediction	Model output	"Our guess" — $\hat{y}$ is always estimated
$e_i = y_i - \hat{y}_i$	Error / residual	How far off we were	"The gap between truth and guess"
w_0	Intercept	Constant term	"Starting point when $x = 0$ "
w_1	Gradient / slope	How much y changes per x	"For every +1 in x , y changes by w_1 "
w	Weight vector	$[w_0, w_1, \dots, w_K]$	"All the model parameters together"
N	Sample count	Number of data points	"How many rows"
K	Number of predictors	How many features	"How many columns (minus the label)"
D	Polynomial degree	Hyperparameter	"How curvy can the line be?"
X	Design matrix	All predictors in matrix form	"The spreadsheet as a matrix (+ 1s column)"
$SSE = \sum e_i^2$	Sum of squared errors	Total squared mistakes	"Add up all squared gaps"
$MSE = SSE/N$	Mean squared error	Average squared mistake	"Average gap squared"
R^2	R-squared	% of variance explained	"How much better than just guessing $\hat{y}$?"

Simple Linear Regression

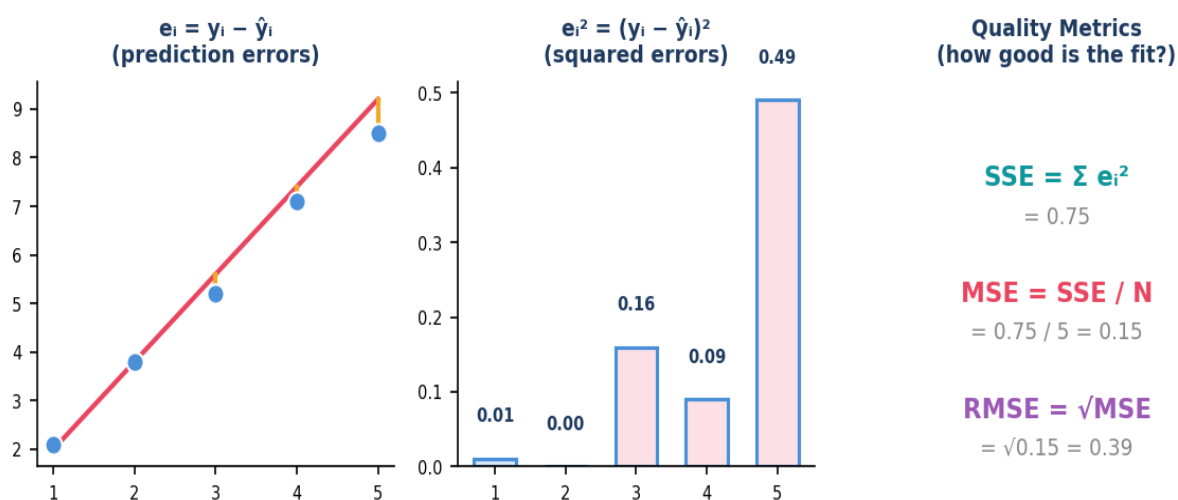


The gold lines are errors (e_i). The model minimises the total squared error. w_0 is where the line starts, w_1 is how steep it is.

Simple Linear Regression — Key Facts

- $\hat{y} = w_0 + w_1x$ → straight line with intercept w_0 and slope w_1
- w_0 (intercept) = predicted value when $x = 0$
- w_1 (gradient) = how much $\hat{y}$ changes for each +1 in x
- $e_i = y_i - \hat{y}_i$ = prediction error for sample i (the vertical gap)
- **Training** = finding w_0 and w_1 that minimise MSE
- The model that minimises MSE is the **least squares solution**

Error, Squared Error & Quality Metrics



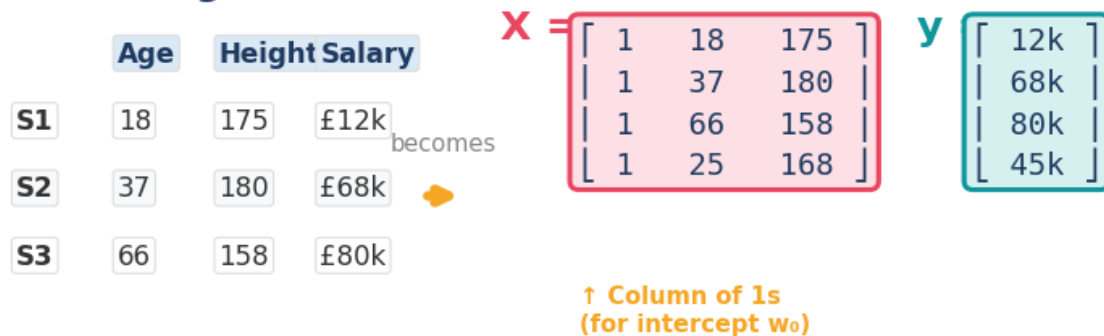
Left: raw errors. Centre: squared errors (always positive). Right: the three quality metrics.

Quality Metrics Decoded

- **SSE** = $\sum e_i^2$ — sum of all squared errors. Bigger = worse fit.
- **MSE** = SSE / N — average squared error. The main quality metric in the lecture.
- **RMSE** = $\sqrt{\text{MSE}}$ — root MSE. Same units as y (e.g. £ not £²). More interpretable.
- **MAE** = $\sum |e_i| / N$ — mean absolute error. Less sensitive to outliers than MSE.
- **R²** = $1 - \text{SSE} / \sum (y_i - \bar{y})^2$ — proportion of variance explained. R²=1 is perfect, R²=0 is useless.
- Why squared? Squaring makes all errors positive AND penalises big errors more heavily.

The Design Matrix: From Spreadsheet to Maths

The Design Matrix — How Your Data Becomes Maths



Your data table becomes matrix X (with a column of 1s prepended for the intercept) and vector y .

■ Matrix Notation — What You Need to Know

- X = design matrix. Each row = one sample. Columns = [1, predictor, predictor, ...]
- The column of 1s is added so that $w \cdot x = w_0 \cdot 1 + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots$ (gives us the intercept)
- y = label vector. All the true values stacked vertically.
- w = parameter vector [$w_0, w_1, \dots, w_n$]. What we're trying to find.
- $\hat{y} = Xw$ — matrix multiplication gives all predictions at once
- $e = y - \hat{y}$ — error vector (all errors at once)

The Least Squares Solution

The Least Squares Solution

$$w_{\text{best}} = (X^T X)^{-1} X^T y$$

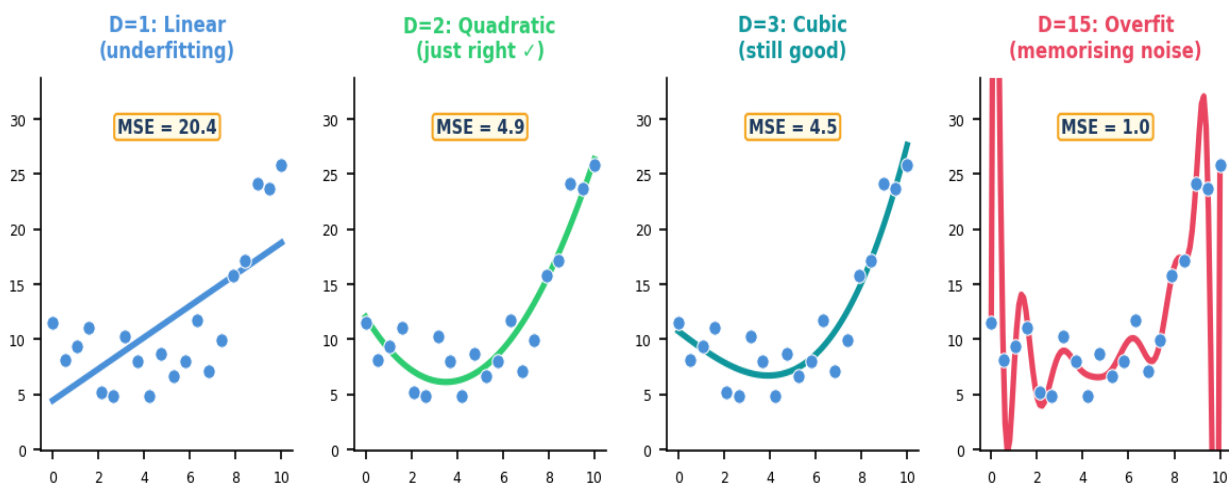
X = design matrix (all predictors + 1s column) $X^T = X$ transposed (rows ↔ columns) $(X^T X)^{-1}$ = inverse (undo the multiplication) y = true labels (what we're fitting to)

This formula gives you the EXACT best weights in one step. No iteration needed.

■ Least Squares — What It Means

- $w_{\text{best}} = (X^T X)^{-1} X^T y$ — the analytical solution for the best linear model
- Works for simple AND multiple regression — same formula
- Also works for polynomial regression (treat x^2, x^3 as extra predictors)
- $X^T = X$ transposed (flip rows and columns)
- $(X^T X)^{-1}$ = inverse of $X^T X$ (only exists when columns of X are independent)
- This is an **exact** solution — unlike gradient descent, no iteration needed

Polynomial Regression: Controlling Flexibility



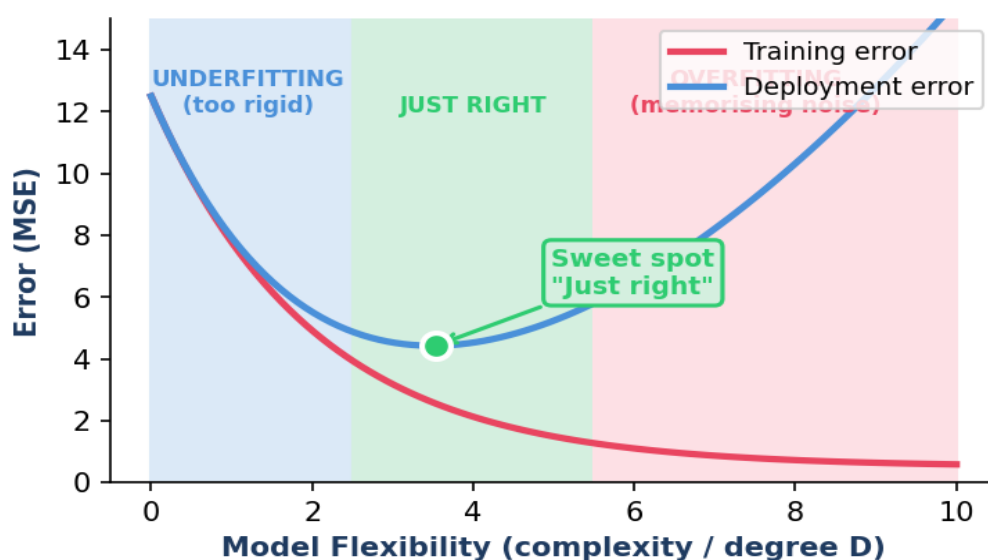
D=1 (linear) underfits. D=2 captures the pattern. D=15 memorises every data point including noise.

Polynomial Regression — Key Facts

- $f(x) = w_0 + w_1x + w_2x^2 + \dots + w_Dx^D$
- **D** is the degree (hyperparameter) — controls flexibility
- D=1 → linear (straight line). D=2 → quadratic (parabola). D=3 → cubic.
- Higher D = more flexible = can fit more complex patterns
- BUT too high D → **overfitting** (memorises noise, fails on new data)
- D is a **hyperparameter** — you choose it BEFORE training. $w_0, w_1, \dots$ are parameters found BY training.

The Flexibility Trade-Off

The Flexibility Trade-Off: Training vs Deployment Error

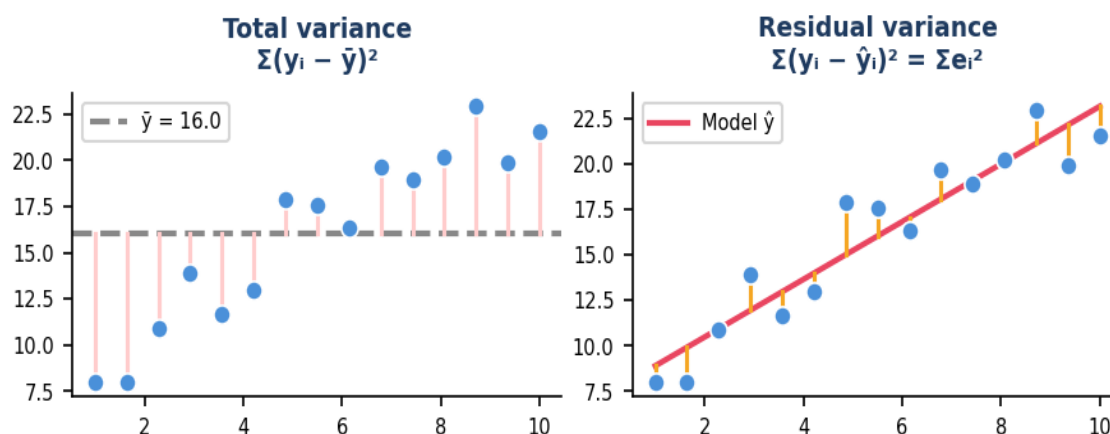


Training error always decreases with flexibility. Deployment error has a U-shape — find the sweet spot.

Underfitting vs Overfitting

- **Underfitting:** Model too simple → can't capture the pattern → high error on BOTH training and deployment
- **Overfitting:** Model too complex → memorises noise → low training error, HIGH deployment error
- **Just right:** Captures pattern, ignores noise → good on both
- **Key insight:** You can ONLY detect overfitting by comparing training AND deployment performance
- Looking at training error alone is MISLEADING — a $D=15$ polynomial has near-zero training error but is terrible

R-Squared: How Good Is Your Model?



$$R^2 = 1 - (\text{residual} / \text{total}) = 1 - 35.4/326.0 = 0.891$$

Left: total variance from the mean $\bar{y}$. Right: residual variance from the model. R^2 = how much the model improves over just guessing $\bar{y}$.

■ R^2 — The Intuition

- $R^2 = 1 - (\text{residual variance} / \text{total variance})$
- $R^2 = 1.0$ → model explains ALL the variance → perfect fit
- $R^2 = 0.0$ → model explains NOTHING → no better than guessing the mean
- $R^2 = 0.85$ → model explains 85% of variance → 15% is unexplained (noise or missing predictors)
- R^2 can be negative if model is worse than just predicting $\bar{y}$ (very bad model)

■ ONE-PAGE NOTATION DECODER

- $\hat{y} = w_0 + w_1x$ → simple linear regression (one predictor)
- $\hat{y} = w_0 + w_1x + w_2x^2 + \dots + w_Kx^K$ → multiple linear regression
- $\hat{y} = w_0 + w_1x + w_2x^2 + \dots + w_Dx^D$ → polynomial regression (degree D)
- $e_i = y_i - \hat{y}_i$ → error | $SSE = \Sigma e_i^2$ | $MSE = SSE/N$ | $R^2 = 1 - \Sigma e_i^2 / \Sigma (y_i - \bar{y})^2$
- $w_{\text{best}} = (X^T X)^{-1} X^T y$ → least squares solution (exact, one step)
- X = design matrix (data + 1s column) | D = degree (hyperparameter) | N = samples | K = predictors

■ EVERY EQUATION DOES ONE OF FOUR THINGS

1. **DEFINE THE MODEL:** What shape? → $\hat{y} = w_0 + w_1x$ (linear) or $+ w_2x^2 + \dots$ (polynomial)
2. **MEASURE ERROR:** How far off? → e_i , SSE, MSE, RMSE, MAE, R^2
3. **FIND BEST PARAMETERS:** Which w minimises MSE? → least squares $(X^T X)^{-1} X^T y$
4. **ASSESS GENERALISATION:** Will it work on new data? → compare training vs deployment error